Reactor Kinetics Lab Physics and Numerical Methods

1 Purpose

This note is the project-wide physics and numerics reference for the current *Reactor Kinetics Lab* repository. It explains how the implemented models are split across:

- the exploratory notebook `theory/reactorModel.ipynb`,
- the point-kinetics web simulation shown on the *Overview* page,
- the steady 2D diffusion solve shown on the *Core* page,
- and the direct time-dependent diffusion solve shown on the *Transient Diffusion* page.

The notebook is an important **calibration and derivation workspace**, while the Python backend and React frontend are the **runtime implementation**. The most important architectural point is:

> The repository now contains **two transient models**: a fast **point-kinetics surrogate** for the Overview page, and a **spatially resolved 2D diffusion transient** for the Transient Diffusion page. Both are tied back to the same one-group annular-core assumptions and the same notebook-derived calibration data.

2 Role of the notebook

The notebook `theory/reactorModel.ipynb` studies a heavy-water moderated annular reactor with cylindrical symmetry in (r, z) and a top-inserted central control/shutdown bank. It is not the simulator itself; instead it is the place where the simplified reactor model is derived, calibrated, and sanity-checked before the resulting constants and algorithms are frozen into the production code. It has two estimation levels.

2.1 Estimate 1: homogeneous buckling sanity check

The first estimate is a coarse analytical model based on one-speed diffusion theory in a homogeneous bare cylinder:

$$-D\nabla^2\phi + \Sigma_a\phi = \frac{1}{k}\nu\Sigma_f\phi.$$

The criticality condition is written as

$$B_m^2 = \frac{\nu\Sigma_f - \Sigma_a}{D} = B_g^2,$$

with geometric buckling for a finite cylinder approximated by

$$B_g^2 \approx \left(\frac{2.405}{R}\right)^2 + \left(\frac{\pi}{H}\right)^2.$$

In the notebook this estimate is used only as a **first-pass size check**. It is intentionally approximate because it ignores:

- the inner moderator hole,
- reflector heterogeneity,
- explicit axial reflector slabs,
- and the geometric control rod.

2.2 Estimate 2: full 2D r–z finite-difference diffusion model

The second estimate is the notebook's main result. It solves a one-group diffusion eigenvalue problem on a cylindrical r–z mesh:

$$-\nabla \cdot (D\nabla\phi) + \Sigma_a\phi = \frac{1}{k_{\text{eff}}}\nu\Sigma_f\phi.$$

This estimate captures:

- radial and axial leakage,
- the annular fuel geometry,
- inner and outer D_2O regions,
- axial reflector slabs,
- and the real rod tip position as insertion changes.

The notebook uses this model to:

1. search for critical annular geometry,
2. compute the clean-core eigenvalue,
3. scan rod insertion versus reactivity,
4. and convert that scan into the calibration constants later frozen in the backend.

3 Geometry, materials, and assumptions

3.1 Core layout

The modeled reactor is an annular cylindrical core with a central D_2O region, a homogenized fuel annulus, and an outer D_2O reflector. The current frozen geometry in `backend/reactor_backend/calibration.py` is:

Quantity	Value
Inner moderator radius R_{inner}	80.0 cm
Fuel outer radius R_{fuel}	345.6 cm
Reflector outer radius R_{refl}	405.6 cm
Active height H	691.2 cm
Axial reflector thickness H_{refl}	60.0 cm per side
Equivalent rod radius r_{rod}	50.0 cm

The model is axisymmetric, so the mesh covers only $r \geq 0$ while still representing the full 3D core through cylindrical volume weighting $2\pi r\, dr\, dz$.

3.2 One-group effective constants

The notebook starts from microscopic cross sections at the 2200 m/s thermal reference point and then builds **engineering-effective** one-group constants. The production code uses the following frozen values:

Region	D [cm]	Σ_a [cm^{-1}]	$\nu\Sigma_f$ [cm^{-1}]
Inner D_2O moderator	1.25	3.5×10^{-4}	0
Fuel annulus	0.95	8.5×10^{-3}	8.59×10^{-3}
Outer D_2O reflector	1.15	4.0×10^{-4}	0

These are **homogenized one-group constants**, not a multigroup transport library. The fuel values intentionally fold together:

- natural uranium fuel,
- D_2O moderation inside the lattice,
- resonance/self-shielding effects,
- and spectrum averaging into one effective thermal group.

The control bank is represented as an added absorber in the inner moderator region with

$$\Delta\Sigma_{a,\text{rod,max}} = 0.25 \text{ cm}^{-1}.$$

This is likewise an effective one-group quantity rather than a detailed rod-cell transport model.

3.3 How the physics was manipulated into these constants

This project deliberately compresses more detailed reactor physics into a one-group annular model that can run interactively. The main reductions are:

1. **Energy collapse.** Multigroup slowing-down and spectral effects are represented by a single thermal group with effective $(D, \Sigma_a, \nu\Sigma_f)$.
2. **Lattice homogenization.** The fuel annulus is not modeled as explicit fuel pins in moderator. Instead natural uranium and D_2O are volume-averaged into one fissile ring region.

3. **Leakage folded into effective losses.** In a one-group model, any loss of neutrons to higher-energy groups or to unresolved geometric detail is represented as part of an effective absorption-like loss.

4. **Equivalent rod model.** The real control/shutdown system is collapsed to one central cylindrical absorber with an effective radius and an effective extra absorption $\Delta\Sigma_a$.

5. **Calibration by 2D diffusion, not by transport.** The final rod worth curve and critical insertion are taken from repeated diffusion eigenvalue solves, then frozen for the point model.

The notebook is explicit about these manipulations. For example, the D_2O "effective" Σ_a is much larger than the true 2200 m/s absorption of pure heavy water, because the one-group model uses Σ_a as a *net loss coefficient* after spectral and geometric simplification. Likewise, the fuel-annulus Σ_a is intentionally inflated relative to a naive 2200 m/s natural-uranium homogenization so the one-group annular model better matches the critical-size and rod-worth behavior seen in the notebook's 2D studies.

3.4 Boundary assumptions

The diffusion solves use extrapolated zero-flux boundaries rather than placing the computational boundary exactly at the physical reflector edge. In code this is represented by an extrapolation factor of $2.13D_{\text{refl}}$, so the radial and axial computational extents are larger than the physical core.

4 How the notebook feeds the web app

The notebook does *not* run the web app directly. Instead it establishes the physical basis that is later frozen into backend constants and reusable solver code:

1. **Estimate 1** gives an analytical reference for rough critical size.

2. **Estimate 2** performs the annular r–z diffusion study that supplies the production calibration.

3. The resulting geometry, material constants, and rod-worth data are copied into `calibration.py`.

4. The backend then exposes two kinds of runtime models:

    - a calibrated point-kinetics model for fast dashboard transients,
    - and spatial diffusion solvers for the Core and Transient pages.

So the notebook is best viewed as the **physics derivation and calibration workspace**, while the Python backend is the **production implementation**.

4.1 How the notebook output becomes backend input

The handoff from notebook to code is not symbolic; it is numerical. The production backend lifts the notebook's final values into `calibration.py`:

- annular geometry dimensions,
- one-group material constants,
- clean-core k_{eff} and excess reactivity,

- the 11-point rod-worth table,
- and the derived critical insertion percentage.

This means the backend no longer recomputes those design-calibration studies on startup. Instead it assumes the notebook has already answered the "what reactor are we modeling?" question and the Python solvers answer the "how does that reactor evolve for this input?" question.

5 Overview page: point-kinetics transient model

5.1 Governing equations

The Overview page uses the standard six-group delayed-neutron point-kinetics system:

$$\frac{dn}{dt} = \frac{\rho - \beta}{\Lambda} n + \sum_{i=1}^{6} \lambda_i C_i, \tag{1}$$

$$\frac{dC_i}{dt} = \frac{\beta_i}{\Lambda} n - \lambda_i C_i, \qquad i = 1, \ldots, 6. \tag{2}$$

Here:

- $n(t)$ is the normalized neutron population,
- ρ is reactivity in $\Delta k/k$,
- $\beta = \sum_i \beta_i$ is the effective delayed-neutron fraction,
- Λ is the prompt neutron generation time,
- C_i is precursor concentration for delayed group i,
- λ_i is the delayed-group decay constant.

5.2 Constants used by the web app

The current production constants in `config.py` are:

$$\beta_{\text{eff}} = 0.00651 = 651 \text{ pcm},$$
$$\Lambda = 5 \times 10^{-4} \text{ s}.$$

The six delayed groups are:

Group	β_i	λ_i [s^{-1}]
1	0.00025	0.0124
2	0.00138	0.0305
3	0.00122	0.1110
4	0.00264	0.3010
5	0.00075	1.1400
6	0.00027	3.0100

Displayed output is scaled from the nominal operating point

$$P_0 = 20 \text{ MW}_{\text{th}}, \qquad \Phi_0 = 1.5 \times 10^{12} \text{ n cm}^{-2} \text{s}^{-1},$$

using

$$P(t) = P_0 n(t), \qquad \Phi(t) = \Phi_0 n(t).$$

5.3 How rod motion enters the point model

The point-kinetics solver does not compute rod worth from first principles during runtime. Instead it reuses the **2D diffusion-derived rod-worth table** frozen in `calibration.py`. Reactivity is computed as

$$\rho_{\text{tot,pcm}}(x) = \rho_{\text{clean,pcm}} + \Delta\rho_{\text{rod,pcm}}(x) + \rho_{\text{scram,pcm}},$$

where $x \in [0, 1]$ is rod insertion fraction. In the current calibration:

$$\begin{aligned} k_{\text{clean}} &= 1.000199, \\ \rho_{\text{clean}} &= +19.9 \text{ pcm}, \\ x_{\text{crit}} &\approx 0.193, \\ \Delta\rho_{\text{rod}}(1.0) &= -164.8 \text{ pcm}, \\ \rho_{\text{scram}} &= -450 \text{ pcm when latched.} \end{aligned}$$

The backend interpolates linearly between 11 precomputed insertion points $x = 0.0, 0.1, \ldots, 1.0$.

5.4 Numerical method

The point-kinetics equations are stiff, so the backend does not use explicit Euler. Instead `engine.py` advances both the neutron population and the precursors implicitly at the new time level and then algebraically eliminates the precursor unknowns.

Start from the backward-Euler discretization

$$\frac{n^{k+1} - n^k}{\Delta t} = \frac{\rho - \beta}{\Lambda} n^{k+1} + \sum_{i=1}^{6} \lambda_i C_i^{k+1}, \tag{3}$$

$$\frac{C_i^{k+1} - C_i^k}{\Delta t} = \frac{\beta_i}{\Lambda} n^{k+1} - \lambda_i C_i^{k+1}. \tag{4}$$

Solving the precursor equation for the new precursor gives

$$C_i^{k+1} = \frac{C_i^k + \Delta t\,(\beta_i/\Lambda)\, n^{k+1}}{1 + \lambda_i \Delta t},$$

which is the relation already visible in the source.

Substituting that expression into the neutron balance yields

$$\frac{n^{k+1} - n^k}{\Delta t} = \frac{\rho - \beta}{\Lambda} n^{k+1} + \sum_i \lambda_i \frac{C_i^k + \Delta t\,(\beta_i/\Lambda)\, n^{k+1}}{1 + \lambda_i \Delta t}.$$

Now separate the terms involving only known old-state data from the terms that still multiply n^{k+1}:

$$\frac{n^{k+1} - n^k}{\Delta t} = \sum_i \frac{\lambda_i C_i^k}{1 + \lambda_i \Delta t} + \left[\frac{\rho - \beta}{\Lambda} + \sum_i \frac{\lambda_i \Delta t\, \beta_i/\Lambda}{1 + \lambda_i \Delta t} \right] n^{k+1}.$$

Rearranging gives one scalar solve for the new neutron population,

$$n^{k+1} = \frac{n^k + \Delta t \sum_i \dfrac{\lambda_i C_i^k}{1 + \lambda_i \Delta t}}{1 - \Delta t \dfrac{\rho - \beta}{\Lambda} - \Delta t \sum_i \dfrac{\lambda_i \Delta t\, \beta_i/\Lambda}{1 + \lambda_i \Delta t}}.$$

In the implementation this matches the code variables `delayed_source`, `delayed_coupling`, `numerator`, and `denominator` in `engine.py`. The precursor update then uses the newly computed n^{k+1}.

Numerically, this matters because the prompt part of the kinetics is very fast relative to the slowest delayed groups. A naive explicit step would require a much smaller Δt for stability, while the semi-implicit update remains well behaved for the chosen browser-friendly step size.

5.5 How the Overview transient is operationalized

The point-kinetics equations are not advanced continuously in a background thread. Instead `service.py` advances them *opportunistically* whenever the frontend polls or sends a command:

1. compute elapsed wall time since the last request,
2. cap that wall-time gap to avoid large jumps after browser pauses,
3. multiply by the configured time-scale factor,
4. then march the engine forward in repeated 0.02 s internal steps.

This architecture is a numerical simplification as much as a software one: the browser sees a continuous transient, but the backend actually performs a sequence of bounded substeps and samples history only every 0.25 s.

The current runtime tuning is:

Parameter	Value
Integrator step	0.02 s
History sampling	0.25 s
Time scale	8× wall clock
Frontend poll interval	100 ms
Automatic SCRAM threshold	26 MW_{th}

6 Core page: steady 2D diffusion solution

The Core page is the repository's **spatial steady-state neutronics view**. It reuses the same annular one-group model as the notebook and solves the 2D eigenvalue problem for the *current rod position* taken from the Overview simulation.

6.1 Discretization

The implementation in `diffusion.py` and `core_service.py` uses:

- a cell-centered finite-difference mesh in r and z,
- harmonic averaging of diffusion coefficients at material interfaces,
- sparse matrix assembly for leakage, absorption, and fission terms,
- and a power-iteration style eigenvalue solve for k_{eff} and the fundamental flux shape.

The production Core page solve runs at

$$\Delta r = \Delta z = 3 \text{ cm},$$

and results are cached by rod insertion state so repeated requests do not repeat the full eigenvalue solve.

6.2 How the steady diffusion matrices are built

The Python solver does not discretize the Laplacian in Cartesian form and then "pretend" the reactor is cylindrical. It assembles the operator directly in cylindrical coordinates. On each cell-centered mesh point (r_i, z_j) it builds:

- radial couplings weighted by the cylindrical face areas $r_{i\pm1/2}$,
- axial couplings weighted by $1/\Delta z^2$,
- and a diagonal absorption term from the local Σ_a.

Material interfaces are handled with the harmonic mean of adjacent diffusion coefficients, which is the standard finite-volume-like choice for discontinuous media because it preserves current continuity better than a simple arithmetic average.

The assembled linear system can be written schematically as

$$L\phi = \frac{1}{k} F\phi,$$

where:

- L is a sparse loss matrix containing leakage and absorption,
- F is a diagonal sparse matrix containing $\nu\Sigma_f$ in fuel cells and zero elsewhere.

The reason for that structure is direct. After cell-centered finite-difference discretization, the loss term at cell (i, j) depends only on:

- the flux in the same cell,
- the two radial neighbors $(i-1, j)$ and $(i+1, j)$,
- and the two axial neighbors $(i, j-1)$ and $(i, j+1)$.

So each row of L contains only a small five-point stencil worth of nonzero entries, making L sparse. By contrast, the fission source has no spatial derivative in this one-group model; it is simply

$$(F\phi)_{ij} = \nu\Sigma_{f,ij}\phi_{ij}.$$

That is a cell-local multiplication, so after flattening the 2D mesh into a single vector, F becomes diagonal.

In other words:

- **leakage/absorption** move neutrons between nearby cells or remove them locally $\rightarrow$ sparse coupling matrix L,
- **fission production** multiplies the flux only in the current cell $\rightarrow$ diagonal production matrix F.

6.3 How the steady eigenvalue solve works numerically

The function `solve_2d()` uses a source-iteration / power-iteration structure:

1. start from a positive flux guess ϕ and an initial $k = 1$,
2. compute the fission source $F\phi$,
3. solve the fixed loss system
$$L\phi^* = F\phi$$
with a sparse direct solver,
4. clip tiny negative roundoff values to zero,
5. normalize ϕ^* by its peak,
6. update the eigenvalue with the ratio of new to old total fission source,
$$k_{\text{new}} = \frac{\sum(F\phi^*)}{\sum(F\phi)},$$
7. and repeat until $|k_{\text{new}} - k| < \texttt{tol}$ after a few warmup iterations.

Two numerical choices are especially important:

- the loss matrix L is factorized once per solve with `scipy.sparse.linalg.factorized()`, so each iteration is just a triangular solve instead of a fresh matrix inversion;
- the flux is normalized by peak value, not by integral power, because the steady eigenproblem only needs the shape and eigenvalue.

6.4 How geometry and rod-worth calibration are solved

The notebook and backend use the steady diffusion solver in two further numerical loops:

1. **critical-radius search:** bisection on R_{fuel} until the solved k_{eff} brackets unity to within tolerance;
2. **rod-worth scan:** repeated solves at insertion fractions $x = 0, 0.1, \ldots, 1.0$, converting each k to reactivity $\rho = (k - 1)/k$ in pcm.

The rod-worth scan may warm-start each solve with the previous flux shape. That is a purely numerical acceleration: neighboring rod positions have similar flux shapes, so reusing the previous converged state reduces iteration count without changing the underlying model.

6.5 What the page returns

The backend returns:

- the peak-normalized 2D flux field $\phi(r, z)$,
- a radial profile through the midplane,
- an axial profile through the fuel-annulus midpoint,
- and metadata including k_{eff}, mesh spacing, and iteration count.

This is the web app's direct counterpart to the notebook's Estimate 2 flux maps.

7 Transient Diffusion page: direct time-dependent diffusion

7.1 Why this page exists

The Overview page assumes a fixed spatial shape and evolves only an amplitude. The Transient Diffusion page removes that simplification and evolves the **spatial flux field itself**, together with the six delayed-neutron precursor fields.

7.2 Governing equations

The transient diffusion solver in `backend/reactor_backend/transient_diffusion.py` solves

$$\frac{1}{v}\frac{\partial \phi}{\partial t} = -L\phi + (1-\beta)\frac{F}{k_{\text{ref}}}\phi + \sum_{i=1}^{6}\lambda_i C_i, \tag{5}$$

$$\frac{\partial C_i}{\partial t} = \beta_i \frac{\nu\Sigma_f}{k_{\text{ref}}}\phi - \lambda_i C_i, \qquad i = 1,\dots,6, \tag{6}$$

where:

- L is the spatial loss operator (leakage plus absorption),
- F is the diagonal fission-production operator,
- $C_i(r,z,t)$ is the precursor field for group i,
- $v = 2.2 \times 10^5$ cm/s is the one-group thermal neutron speed,
- k_{ref} is the eigenvalue of the initialization state.

The division by k_{ref} is deliberate: it references the transient fission source to the eigenvalue of the initialized state on the transient mesh. That keeps the transient solve anchored to the same discrete criticality reference and substantially reduces startup mismatch relative to using the raw unscaled fission operator.

7.3 Numerical method

The transient page uses **fully implicit Euler**. Eliminating the precursors gives one sparse linear solve per time step:

$$A\phi^{n+1} = b,$$

with

$$A = \frac{1}{v\Delta t}I + L - b_{\text{eff}}\,\text{diag}\left(\frac{\nu\Sigma_f}{k_{\text{ref}}}\right),$$

and

$$b_{\text{eff}} = (1-\beta) + \sum_i \frac{\beta_i \lambda_i \Delta t}{1+\lambda_i \Delta t}.$$

After ϕ^{n+1} is solved, each precursor field is updated by

$$C_i^{n+1} = \frac{C_i^n + \Delta t\,\beta_i(\nu\Sigma_f/k_{\text{ref}})\phi^{n+1}}{1+\lambda_i\Delta t}.$$

This scheme is unconditionally stable for the linear model and is much better suited than explicit Euler to delayed-neutron reactor kinetics.

In implementation terms, the transient solver performs three numerically distinct operations:

1. **Initialization:** run a steady `solve_2d()` eigenvalue solve on the transient mesh, flatten the 2D flux field, and initialize each precursor field with the steady relation

$$C_{i,0} = \frac{\beta_i}{\lambda_i} \frac{\nu\Sigma_f}{k_{\text{ref}}} \phi_0.$$

2. **Matrix rebuild on rod motion:** if rod insertion changes, rebuild the sparse matrix A and refactorize it. In code this cache is keyed by the rounded rod fraction together with k_{ref}.

3. **Cheap repeated stepping between rod changes:** if rod insertion is unchanged, reuse the cached LU factorization so each new time step is just one sparse linear solve plus explicit vector updates for the six precursor groups.

This split is why the transient page can remain interactive even though it is a full spatial solver: the expensive part is the factorization, not the individual post-factorization time steps.

7.4 How the transient physics is manipulated into a solvable model

The transient page is more detailed than point kinetics, but it is still not a full reactor multiphysics model. Several physics reductions are built into the solver:

- prompt and delayed neutrons are treated in one energy group,
- thermal-hydraulic feedback is absent, so all cross sections stay fixed while the transient runs,
- rod motion changes only the absorber map inside the central control-bank cylinder,
- the internally stored flux is an *absolute amplitude field*, while UI heatmaps are separately peak-normalized display copies,
- and power is derived from a cylindrical fission integral rather than from a standalone thermal-hydraulic power balance.

That last point is numerically important: the solver never renormalizes its internal state after each step. If it did, the transient could not display real growth or decay in P/P_0.

7.5 Runtime settings

To keep the direct transient responsive in a browser-driven local app, the production solver uses:

Parameter	Value
Transient mesh	$\Delta r = \Delta z = 5$ cm
Time step	$\Delta t = 1$ s
Display heatmap	40×60 points
History limit	200 points

The transient mesh is intentionally coarser than the Core page mesh so the LU factorization can be rebuilt quickly whenever rod insertion changes.

7.6 Service-level time stepping

The transient page is advanced by `transient_service.py`, which keeps one shared solver instance and one shared state vector. On every `GET /api/transient-diffusion/state` request it:

1. measures elapsed wall time,
2. converts that to simulated time at a fixed scale of $1\times$,
3. computes how many whole 1 s transient steps fit in that interval,
4. caps the advance to at most four steps per request,
5. and appends one history sample per accepted time step.

This cap is not part of the physics model; it is a numerical and UX safeguard so one delayed frontend poll cannot trigger an unexpectedly long blocking solve.

7.7 Power normalization

The transient diffusion page no longer assumes that power is proportional to one global amplitude alone. Instead it evaluates the cylindrical fission-rate integral

$$P \propto \sum_j \nu\Sigma_{f,j}\,\phi_j\,V_j, \qquad V_j \propto r_j \Delta r \Delta z,$$

and reports

$$\frac{P}{P_0} = \frac{\sum_j \nu\Sigma_{f,j}\phi_j V_j}{\sum_j \nu\Sigma_{f,j}\phi_{j,0} V_j}.$$

The factor 2π cancels in the ratio, so the implementation only needs the $r_j \Delta r \Delta z$ weight per cell.

8 How the three app pages relate

Layer	Model	Purpose
Notebook estimate 1	Homogeneous buckling	First-pass analytical check of scale and leakage trends
Notebook estimate 2 / Core page	Steady 2D one-group diffusion	Compute k_{eff}, flux maps, and rod-worth calibration
Overview page	0D six-group point kinetics	Fast transient driven by the calibrated reactivity curve
Transient Diffusion page	2D diffusion + 6 precursor fields	Spatial transient with evolving flux shape and power

The conceptual workflow is therefore:

1. derive or justify effective constants in the notebook,
2. solve the steady annular diffusion problem and calibrate rod worth,

3. freeze those results into backend constants,
4. use the fast point model for dashboard-style operation,
5. and use the diffusion transient when spatial dynamics matter.

9 Important assumptions and limitations

The current model remains intentionally simplified:

- one energy group only,
- no thermal-hydraulic feedback,
- no fuel temperature, moderator temperature, void, xenon, or burnup effects,
- homogenized fuel and rod regions,
- heavy-water moderation is represented through effective diffusion and absorption constants rather than coupled moderator state equations.

So the repository should be read as an **educational reactor simulator with increasing levels of neutronics fidelity**, not as a licensing-grade core code.

10 Bottom line

The repository currently implements a hierarchy of reduced reactor models with increasing spatial fidelity. The notebook remains the physics workbench that derives the one-group annular-core picture, performs the 2D criticality and rod-worth studies, and helps justify the frozen constants used by the backend.

The web app then uses that physics in two ways:

- as a reduced **point-kinetics transient** for fast control-response exploration on the Overview page,
- and as a direct **diffusion-based spatial solve**, first in steady form on the Core page and then in time-dependent form on the Transient Diffusion page.

That is the current modeling hierarchy of the project.